



Objectifs

- Apprendre à ranger les fichiers d'un projet Java
- Apprendre à compiler, exécuter et générer la javadoc **en lignes de commande**
- Implémenter une application simple avec une architecture MVC

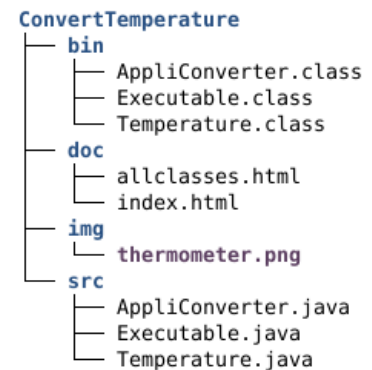


Dans cette feuille, la compilation du code, l'exécution du programme et la génération de la javadoc se fera en lignes de commandes

Exercice 1 *Ranger ses fichiers*

Tu as maintenant l'habitude de créer un dossier pour chaque projet Java que tu développes. Il est temps d'aller un peu plus loin et de prendre l'habitude de ranger les fichiers dans des dossiers. Par exemple :

- Les fichiers `.java` dans un dossier `src`
- Les fichiers `.class` dans un dossier `bin`
- La documentation dans un dossier `doc`
- Les images dans un dossier `img`
- ... etc.



1.1 Récupère le projet *Temperature (sans IHM)* sur Celene.

1.2 Dans un dossier `ConvertTemperature`, crée les dossiers nécessaires et range les fichiers au bon endroit. Pour le moment, certains dossiers sont vides : c'est normal !

1.3 Lis le code fourni et décris en quelques mots sur la feuille réponse ce que fait ce programme.

Exercice 2 *Compiler, exécuter et générer la javadoc en ligne de commande (sans JavaFX)*

Pour **compiler** les fichiers sources (qui doivent être dans le dossier `src`) et faire en sorte que les fichiers générés se mettent dans le dossier `bin`, on utilise l'option `-d`.

Pour cela, place toi à la racine de ce projet, c'est à dire dans le dossier `ConvertTemperature` puis tape la commande suivante :

```
javac -d bin src/*.java
```

2.1 Compile les deux fichiers du projet et vérifie que les fichiers `.class` ont bien été générés dans le dossier `bin`.

Pour **exécuter** le programme, il faut préciser le *classpath* c'est à dire les dossiers dans lesquels se trouvent les différentes ressources (les classes mais aussi images si besoin). Ici, les ressources nécessaires sont les fichiers `.class` qui sont dans le dossier `bin`.

Place toi à la racine de ce projet, c'est à dire dans le dossier `ConvertTemperature` puis tape la commande suivante :

```
java -cp bin Executable
```

2.2 Exécute le programme et vérifie ta réponse à la question **1.3**.

Pour **générer la javadoc**, réfère toi à la feuille de TP précédente. Inutile ici de préciser où se trouve les librairies JavaFX puisqu'on ne les utilise pas. En revanche, il faut spécifier le chemin vers les fichiers source.

```
javadoc -d doc src/*.java
```

ou encore avec quelques options :

```
javadoc -d doc -charset utf8 -noqualifier all src/*.java
```

2.3 Génère la javadoc et vérifie qu'elle est correctement créée.

2.4 Réponds au Quizz sur CELENE pour vérifier tes connaissances et débloquent la suite du TP. Si tu as l'impression de ne pas avoir bien compris certaines notions, tu DOIS poser des questions à ton enseignant.

Exercice 3 *Compiler, exécuter et générer la javadoc en ligne de commande (avec JavaFX)*

Il est temps d'ajouter une interface graphique. Les commandes `javac`, `java` et `javadoc` vont donc avoir besoin des librairies JavaFX. Pour cela, on ajoute les deux options :

```
--module-path /usr/share/openjfx/lib/ et --add-modules javafx.controls.
```

3.1 Récupère le projet *Temperature (avec IHM)* sur Celene et range les fichiers au bon endroit.

3.2 Compile les fichiers source et vérifie que les fichiers `.class` ont bien été générés dans le dossier `bin`.

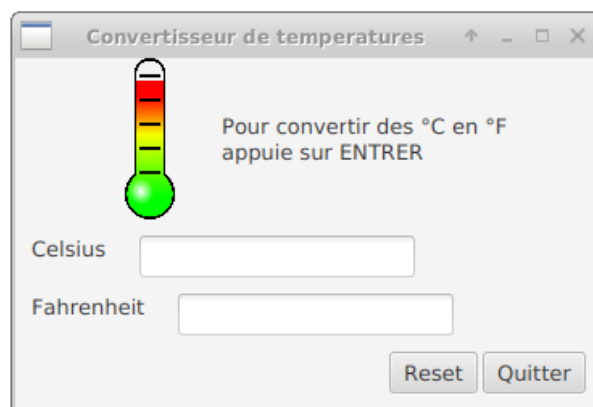
3.3 Génère la javadoc et vérifie qu'elle a bien été générée.

Pour **exécuter** le programme, il faut compléter le *classpath*. En effet, les ressources nécessaires au bon fonctionnement de l'application sont dans deux dossiers différents : le dossier `bin` qui contient les fichiers `.class` et le dossier `img` qui contient l'image.

```
java -cp bin:img --module-path /usr/share/openjfx/lib/ --add-modules javafx.controls AppliConverter
```

Tu noteras la syntaxe : les dossiers du *classpath* sont séparés par `:` sans espace.

3.4 Exécute l'application et manipule-la quelques instants.



3.5 Construis (sur la feuille réponse) le diagramme de cas d'utilisation de cette application.

Remarque : cette application est très simple. Le diagramme CU doit l'être aussi

Exercice 4 Lecture de code

4.1 A l'aide des codes source et de la javadoc, complète le diagramme de classe donné sur la feuille réponse avec des associations unidirectionnelles¹.

4.2 Identifie le module (Modèle, Vue ou Controleur) de chacune des classes.

Exercice 5 Ajout d'une fonctionnalité

On veut ajouter un bouton `Convertir °C` qui met à jour le contenu du champ *Fahrenheit* en fonction de contenu du champ *Celsius*.

5.1 Quels modules (Modèle, Vue ou Controleur) doivent être modifiés ?

5.2 Ajoute le bouton dans le fichier `AppliConverter`. Ce bouton est inactif pour le moment.

5.3 Crée un nouveau contrôleur qui va permettre de rendre le bouton fonctionnel.

5.4 Complète le code de façon à ce que l'application soit totalement fonctionnelle.

5.5 Ajoute un bouton `Convertir °F` qui met à jour le contenu du champ *Celsius* en fonction de contenu du champ *Fahrenheit*.

Exercice 6 Ajout d'une autre fonctionnalité

On veut maintenant ajouter les degrés Kelvin à l'application.

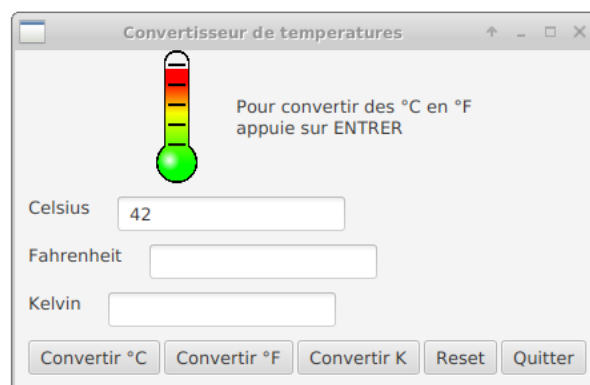
6.1 Quels modules (Modèle, Vue ou Controleur) doivent être modifiés ?

6.2 Commence par compléter le fichier `Temperature.java`

6.3 Ajoute ensuite les éléments graphiques dans le fichier `AppliConverter`. Ces éléments sont inactifs pour le moment.

6.4 Crée un nouveau contrôleur qui va permettre de rendre les éléments graphiques ajoutés actifs.

6.5 Complète le code de façon à ce que l'application soit totalement fonctionnelle.

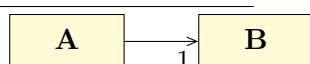


Exercice 7 Amélioration de l'ergonomie

7.1 Comment pourrait-on améliorer l'ergonomie de cette application ?

Les réponses sont attendues sur la feuille réponse

1. Petit rappel :

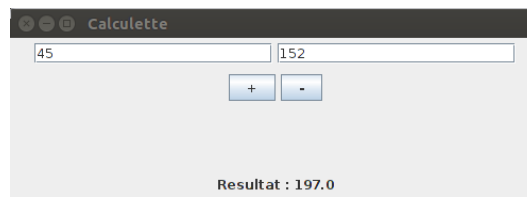


signifie

A possède 1 attribut de type B

Exercice 8 *Calculatrice*

Tu dois implémenter une calculatrice dont la maquette est donnée en respectant le pattern MVC.



8.1 Conçois un diagramme de classes en précisant :

- pour chaque classe si elle appartient au Modèle, à la Vue ou au Contrôleur.
- les attributs de chaque classe
- les méthodes

Réponses attendues sur la feuille réponse

8.2 Implémente le projet (pour le moment, on ne se préoccupe pas du positionnement des widgets)

8.3 On peut améliorer l'application en gérant les exceptions. Précise quelles sont les exceptions qu'il est utile de gérer. Quel traitement apporter ?

Réponses attendues sur la feuille réponse

8.4 Implémente ta solution.

Exercice 9 *Calculatrice - pour aller plus loin*



Fais évoluer ta calculatrice de façon à :

- pouvoir faire les quatre opérations
- ajouter un historique des opérations effectuées

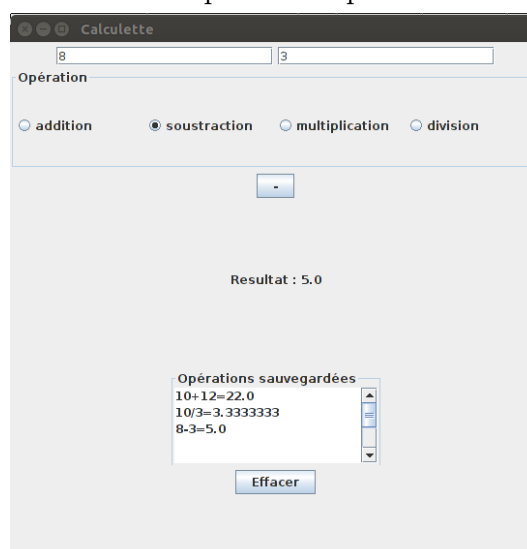
9.1 Reprends ton diagramme de classes pour le compléter

9.2 Imagine une maquette qui pourrait convenir

9.3 Implémente ta solution

Pour le moment, on ne se préoccupe pas du positionnement des widgets : ce sera l'objet d'étude la semaine prochaine.

Exemple de maquette



Nom: _____

Groupe: _____

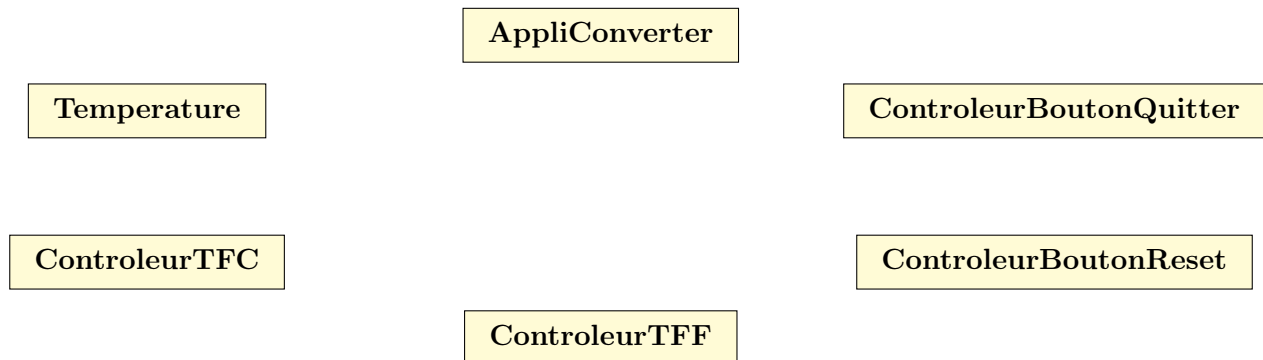
Feuille réponse

Exercice 1

Réponse :

Diagramme de Cas d'Utilisation

Diagramme de classes (très simplifié)



Améliorations envisagées

Réponse :

Exercice Calculatrice (réponses à écrire au verso de la feuille)